HW06

Sam Ly

March 12, 2026

Chapter 12

https://github.com/samlyme/Assignments/tree/cs4080/ch12/challenge

1. We have methods on instances, but there is no way to define "static" methods that can be called directly on the class object itself. Add support for them. Use a class keyword preceding the method to indicate a static method that hangs off the class object.

```
class Math {
  class square(n) {
    return n * n;
  }
}

print Math.square(3); // Prints "9".
```

As suggested, this was implemented with metaclasses. When parsing, we perform a simple `match(CLASS)` to mark the function as being a static method.

This allows for us to create another `LoxClass` object, that has references to the static methods. The bulk of the work happens at the interpreter and `LoxClass` implementation.

```
        LoxClass metaKlass = new LoxClass(
            null,
            stmt.name.lexeme + " meta",
            staticMethods
        );
        LoxClass klass = new LoxClass(metaKlass, stmt.name.lexeme, methods);
```

```
class LoxClass extends LoxInstance implements LoxCallable {
    final String name;
    private final Map<String, LoxFunction> methods;

    LoxClass(LoxClass metaClass, String name, Map<String, LoxFunction> methods) {
        super(metaClass);
        this.name = name;
        this.methods = methods;
    }
// ...
```

This alows for the `LoxInstance` object to resolve the static methods.

2. Most modern languages support "getters" and "setters"—members on a class that look like field reads and writes but that actually execute user-defined code. Extend Lox to support getter methods. These are declared without a parameter list. The body of the getter is executed when a property with that name is accessed.

```
class Circle {
  init(radius) {
    this.radius = radius;
  }

  area {
    return 3.141592653 * this.radius * this.radius;
  }
}

var circle = Circle(4);
print circle.area; // Prints roughly "50.2655".
```

This can be done by having the `visitGetExpr` method intercept getter functions. We simply have the getter function be a special type of function that has an `isGetter` field set to true. This way, the visitor function can do an if statement and evaluate the function.

```
    public Object visitGetExpr(Expr.Get expr) {
        Object object = evaluate(expr.object);
        if (object instanceof LoxInstance) {
            Object val = ((LoxInstance) object).get(expr.name);
            if (val instanceof LoxFunction) {
                if (((LoxFunction) val).isGetter()) {
                    return ((LoxFunction) val).call(this, new ArrayList<>());
                }
            }
            return val;
        }

        throw new RuntimeError(expr.name,
                "Only instances have properties.");
    }
```

3. Python and JavaScript allow you to freely access an object's fields from outside of its own methods. Ruby and Smalltalk encapsulate instance state. Only methods on the class can access the raw fields, and it is up to the class to decide which state is exposed. Most statically typed languages offer modifiers like private and public to control which parts of a class are externally accessible on a per-member basis.

   What are the trade-offs between these approaches and why might a language prefer one or the other?

   The only benefit afforded by Python and JavaScript's appraoch is simplicity of implementation. In my opinion, access control modifiers are a 'pure upgrade' from a language point of view, however can be a pain to implement. In fact, in the world of Python development, they invented their own form of private fields via 'name-mangling', ie. adding a series of _ characters to the front of the field's name, so that the property is hard to find, and won't be unintentionally updated.

   This shows that access modifiers are almost always a plus in languages that provide object orientation.